Lecture 7
Pull Requests, Forks, and Code Review

Proposing a change, discussing it, merging it, and working on other people's projects

Learning goals

- Explain what a pull request is and walk through its life from branch to merge.
- Open a pull request, and review one: comments, suggestions, approve, request changes.
- Choose between GitHub's three merge buttons.
- Contribute to a project you do not own: fork, clone, `upstream`, and keeping your fork in sync.
- Inspect and test a teammate's branch on your own computer.

1 What is a pull request?

In Lecture 5 you merged a branch yourself, with one command. On a team, you usually should *not* do that unilaterally: someone else should look at the change first. A **pull request** (PR) is a *request to merge a branch*, shown as a web page where people can read the changes, discuss them, and finally merge them.

Key idea

A pull request is a GitHub feature, not a Git command. Git only knows branches and merges. GitHub adds the page around a branch: the list of changes, the conversation, the approvals, and a button to merge. The name comes from the project's *maintainer* (the person who looks after it) *pulling* your work into their repository.

The whole life of a change on a team looks like this:

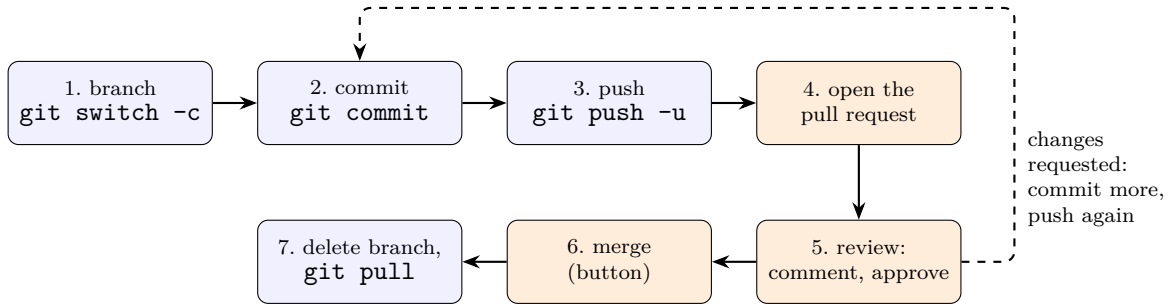


Figure 1: The pull-request workflow. Orange steps happen on the website; blue steps are the Git commands you already know. If the reviewer asks for changes (dashed), you commit more on the same branch and push; the pull request updates by itself.

2 Opening a pull request

Suppose you work in a repository you may push to: your own, or a team's where you are a *collaborator* (someone the owner invited under Settings → Collaborators; the invitation must be accepted). Everyone else cannot push, and uses a *fork* instead (Section 7). Create a branch, commit, and push it, exactly as in Lecture 6:

```
$ git switch -c add-note
$ echo "Remember to drink water." > note.md
$ git add note.md
$ git commit -m "Add a hydration note"
$ git push -u origin add-note
```

Now open the repository on github.com. Right after the push a yellow banner appears with a button **Compare & pull request** (otherwise: **Pull requests** → **New pull request**). The form asks:

- **base** (the branch you want to merge *into*, usually `main`) and **compare** (your branch; GitHub also calls it the *head*). Check them: "merge `add-note` into `main`".
- A **title**, in the style of a commit message: short and imperative.
- A **description**: *what* you changed and *why*, and anything the reviewer should look at. (Use `Closes #12` to link an issue; Lecture 9.)

Press **Create pull request**. You get a page with the conversation, the list of commits, and a **Files changed** tab. That tab is the same comparison as `git diff main...add-note` on your computer.

Quick fact

The three dots in `git diff main...add-note` mean "the changes on `add-note` since it split off from `main`": your work, without whatever else happened on `main` meanwhile. (For `git log`, `main..add-note` with two dots means "the commits on `add-note` that `main` lacks".)

Quick fact

A good pull request is *small* (one idea, a few files) with a clear title and description. Reviewers read small requests carefully and big ones badly. If your work is not ready, choose **Create draft pull request**: it cannot be merged until you press **Ready for review**.

3 Reviewing a pull request

A **review** is a colleague reading your change before it joins `main`. Reasons: catching mistakes, sharing knowledge, keeping the code consistent. Nobody is perfect, and fresh eyes find typos you cannot see.

As a reviewer, open the **Files changed** tab:

- Hover over a line number and click the blue + to add an *inline comment* on that line.
- Instead of describing a fix, you can **suggest** it: the *Add a suggestion* button in the comment box inserts a block with the corrected text. The author can accept it with one click.
- When you have read everything, press **Review changes**, choose one of three verdicts, and **Submit review**:

Verdict	Meaning
Comment	General feedback, no decision.
Approve	"This is good to merge."
Request changes	"Please fix the points I raised first."

You cannot approve your own pull request, which is why review needs two people. As the author, you answer in the conversation (a reviewer's suggestion is applied with **Commit suggestion**) and *fix by committing*: make the change on your branch and push it. You do not open a new request; the same pull request picks up the new commits. When a comment is dealt with, mark the conversation as resolved.

Key idea

Review the *code*, never the person. Be specific ("this variable name is confusing: maybe `total_price`?"), explain why, say what is good too, and ask questions rather than issue orders. When you receive a review, remember that every comment is about the change, not about you.

4 Keeping your pull request mergeable

Conflicts. If someone merged other changes meanwhile, GitHub may say *This branch has conflicts that must be resolved.* Solve it on your computer: merge `main` into your branch, resolve the conflict (Lecture 5), commit and push; the pull request updates.

```
$ git switch add-note
$ git fetch origin
$ git merge origin/main        # fix the conflict if there is one, then commit
$ git push
```

Pitfall

Do not start your branch from an old `main`. Before a new branch, run `git switch main`, `git pull`, and only then `git switch -c new-branch`. Otherwise you carry old code and invite conflicts.

5 Merging: three buttons

When the review is positive, the green merge button offers three ways to merge:

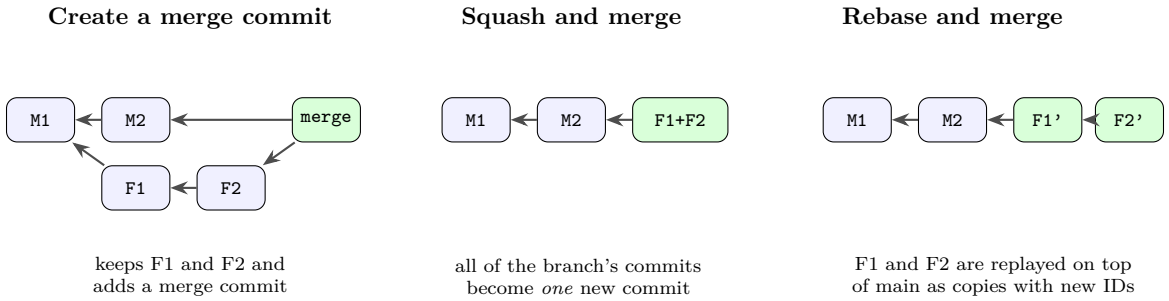


Figure 2: The three merge buttons. `M1`, `M2` are on `main`; `F1`, `F2` are the commits of the pull request's branch. Green commits are new.

Create a merge commit
The merge you know from Lecture 5. Keeps every commit and the real history. A safe default.

Squash and merge
Combines all commits of the branch into one. Good when the branch has messy "fix typo" commits and you want a tidy `main`. The branch's own commits are not kept in `main`.

Rebase and merge
Replays the commits on top of `main` without a merge commit, giving a straight line. Lecture 8

explains rebasing; for now just know it exists.

After merging, press **Delete branch**: it has done its job (Lecture 5's life cycle). Then bring the result to your computer and tidy up:

```
$ git switch main
$ git pull
$ git branch -d add-note        # fine after a merge commit
```

(After *Squash and merge* or *Rebase and merge*, GitHub created *new* commits, so Git cannot see that your branch's commits are included. Depending on whether Git still has the old `origin/add-note` label, `-d` then either refuses ("not fully merged") or succeeds with a warning. If it refuses and you checked on GitHub that the pull request is merged, use `git branch -D add-note`. This is one of the rare safe uses of `-D`: you checked on GitHub that the pull request was merged.)

6 Looking at a teammate's branch

Looking at a teammate's branch before you review it. Fetch, look, and (if you want to run it) check it out:

```
$ git fetch
$ git log --oneline main..origin/teammate-feature     # commits it adds
6bce7d7 Document greeting
14ff5f7 Add greeting
$ git diff --stat main...origin/teammate-feature     # files it changes
 README.md    | 1 +
 greeting.txt | 1 +
 2 files changed, 2 insertions(+)
$ git switch teammate-feature
branch 'teammate-feature' set up to track 'origin/teammate-feature'.
```

`git switch` on a name that exists only on the remote creates a local branch that tracks it. If you have write access you may push a fix to that same branch; the open pull request then shows it.

7 Forks: contributing to a project you do not own

You cannot push to someone else's repository (nor should you be able to). Open-source projects therefore use **forks**. A fork is your own copy of the repository, *on GitHub, under your account.* You push to your fork, and then ask the owner to pull from it.

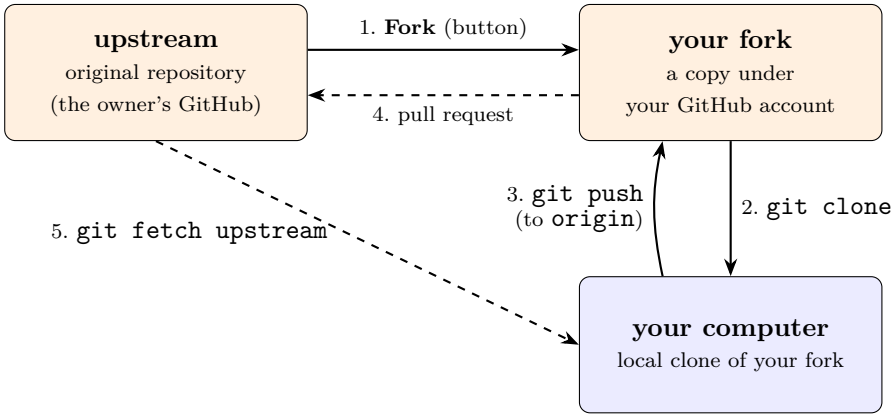


Figure 3: Three copies. You clone your fork (`origin`) and also add the original as a second remote called `upstream`, to follow its progress.

1. On the original repository's page click **Fork**, then **Create fork**.
2. Clone *your fork* (not the original) and tell Git about the original:

```
$ git clone git@github.com:YOU/project.git
$ cd project
$ git remote add upstream git@github.com:OWNER/project.git
$ git remote -v
origin    git@github.com:YOU/project.git (fetch)
origin    git@github.com:YOU/project.git (push)
upstream  git@github.com:OWNER/project.git (fetch)
upstream  git@github.com:OWNER/project.git (push)
```

`origin` is your fork, where you may push. `upstream` is the original, the usual name for it, where you only *read*. (Do not confuse this remote *name* with the "upstream branch" that `-u` set up in Lecture 6.) Work as before: a branch per change, push it to `origin`, and open the pull request. On the pull-request page, the *base repository* is the owner's, and the *head repository* is your fork.

7.1 Keeping your fork up to date

The original keeps moving. To catch up, fetch from `upstream` and merge into your `main`:

```
$ git fetch upstream
$ git switch main
$ git merge upstream/main          # usually a fast-forward
$ git push origin main             # update your fork on GitHub
```

Key idea

Never commit your own work directly on the fork's `main`; make a branch. Then `main` stays a clean mirror of `upstream/main`, and catching up is always a simple fast-forward. (GitHub also offers a **Sync fork** button on your fork's page, which does the same thing on the website.)

8 Homework 7

Homework 7: Review and merge

Part A: be the reviewer and the maintainer (no internet needed). The setup script creates `practice/hw7_server.git`, a bare repository playing the role of GitHub. A teammate has pushed a branch `teammate-feature` and wants it merged into `main`. From the course folder:

```
$ bash codeHints/L07/setup_hw7.sh
$ cd practice
```

Do the tasks in order. To start over: `setup_hw7.sh --reset` from the course folder.

7.1 Clone

Clone the server into a folder called `hw7` and go into it.

7.2 Review

Run `git fetch`, then look at the commits the branch adds and at its changes, using the commands of Section 6. One line contains a typo. In `practice/hw7_answers.txt` (next to the `hw7` folder, created with `printf` or `echo`) write two lines: `commits=<number of commits on the branch>` and `typo-in=<file name containing the typo>`.

7.3 Suggest a fix

Check out the branch (`git switch teammate-feature`), fix the typo, commit it as `Fix typo in greeting`, and push to the same branch with `git push`.

7.4 Merge

Switch to `main` and merge the branch *with a merge commit*: `git merge --no-ff --no-edit teammate-feature`. Push `main`.

7.5 Clean up

Delete the branch locally (`git branch -d`) and on the server (`git push origin --delete teammate-feature`).

7.6 Question

On GitHub, which of the three merge buttons would produce the same result as your command in 7.4 (all commits kept, plus a merge commit)? Add the line `merge-button=<word>` with `merge`, `squash` or `rebase`.

Part B: real pull requests with your friend (optional, but this is the point of the lecture). You need two GitHub accounts, called **A** and **B**. Before you start: both repositories must be created as **Public** (not Private), because the checker looks them up without logging in; and if you ticked "Block command line pushes that expose my email" in Lecture 6, make sure `git config user.email` in your clone is your no-reply address. The two rounds are done one after the other. *Each of you ends up with exactly one clone, `practice/hw7_pr`*: B's from Round 1, A's from Round 2 (it is not the same as `practice/hw7` of Part A). Run the clone commands from the course folder.

7.7 Round 1: shared repository, B's pull request

A creates a *Public* repository `pr-practice` with a README (ticking "Add a README" is fine here), and adds **B** under Settings → Collaborators → Add people (type B's exact user name). **B** *must accept the invitation*: open the e-mail from GitHub, or go to `github.com/A/pr-practice/invitations`. Until B accepts, a push fails with "Permission denied" or "403". Then **B** clones into `practice/hw7_pr`, creates the branch `add-b-note` with a new file `notes-from-b.md`, pushes it, and opens a pull request with a title and a description. **A** reviews: at least one inline comment (use *Add a suggestion* once), then *Submit review* with *Request changes*. **B** responds (clicking *Commit suggestion*, or committing a fix on the

same branch and pushing). **A** approves and merges with *Create a merge commit*. **B** presses *Delete branch*, then in the clone: `git switch main`, `git pull`, `git branch -d add-b-note`.

7.8 Round 2: fork, A's pull request

Swap roles. **B** creates a *Public* repository `pr-practice-b` with a README. **A** forks it and clones *the fork* into `practice/hw7_pr`, runs `git remote add upstream` with B's repository address, creates the branch `add-a-note` with a new file, pushes it to the fork, and opens a pull request *to B's repository* (check that the base repository is B's). **B** reviews with a suggestion and requests changes. **A** clicks *Commit suggestion*. **B** approves and merges with *Squash and merge*. **A** syncs the fork: `git fetch upstream`, `git switch main`, `git merge upstream/main`, `git push origin main`. (A's local `add-a-note` may be behind GitHub's copy now; harmless, since A only syncs `main`.)

7.9 Record and check

Each of you records the pull request *you opened*: **B** the one from Round 1, **A** the one from Round 2. In `practice/hw7_pr_answers.txt` write one line `my-pr=<owner>/<repository>#<number>`, for example `my-pr=alice/pr-practice#1`. Read the number from the top of the pull request page (a second attempt may be `#2`). Be on `main` with a clean working directory, and run the checker.

Check yourself: `bash codeHints/L07/check_hw7.sh` from the course folder. Part A needs no network. Part B is checked only if `practice/hw7_pr` or `hw7_pr_answers.txt` exists; the checker asks GitHub's public service whether your pull request is merged, and skips that one check if it cannot reach GitHub. Then compare with the solutions at the end of this lecture.

9 Summary

- A pull request is a reviewable proposal to merge a branch. Branch, commit, push, open, review, merge, delete, pull.
- Reviewers comment, suggest, approve or request changes; authors answer by pushing more commits to the same branch.
- Merge with a merge commit (keeps everything), squash (one commit) or rebase (straight line); then delete the branch.
- Fork what you cannot push to: clone the fork as `origin`, add the original as `upstream`, branch for each change, merge `upstream/main` to stay current.

Further reading: *Pro Git*, Chapter 6.2; GitHub Docs, "About pull requests".

Solutions

Stop here if you have not tried the homework yet.
Attempt every task first, run the checker, and only then read on.

Solutions to Homework 7

Part A.

```
$ bash codeHints/L07/setup_hw7.sh
$ cd practice
$ git clone hw7_server.git hw7
$ cd hw7
$ git fetch
$ git log --oneline main..origin/teammate-feature     # 2 commits
$ git diff main...origin/teammate-feature             # "Helo, world" in greeting.txt
$ printf 'commits=2\ntypo-in=greeting.txt\nmerge-button=merge\n' > ../hw7_answers.txt
$ git switch teammate-feature
$ sed -i.bak 's/Helo/Hello/' greeting.txt && rm greeting.txt.bak    # or edit by hand
$ git commit -am "Fix typo in greeting"
$ git push
$ git switch main
$ git merge --no-ff --no-edit teammate-feature
$ git push
$ git branch -d teammate-feature
$ git push origin --delete teammate-feature
```

7.2. The branch adds two commits (`Add greeting` and `Document greeting`). The typo is `Helo` in `greeting.txt`. (`git diff` with three dots shows only what the branch added.)

7.3. `git switch teammate-feature` finds no local branch of that name, but there is `origin/teammate-feature`, so Git creates a tracking branch for you. A plain `git push` then updates the remote branch, as when you push a fix to an open pull request.

7.4. `--no-ff` forces a merge commit even though a fast-forward would be possible. This is what GitHub's button *Create a merge commit* does, hence the answer `merge`. (The checker expects a merge commit at the tip of the server's `main`; a fast-forward would fail.)

7.5. `git branch -d` works because the branch is merged. `git push origin --delete` removes it on the server (on GitHub, the *Delete branch* button).

Part B. Replace `A`, `B` by your account names.

Round 1: shared repository (B opens the pull request).

Who	Step
A	Create a *public* repository `pr-practice` with a README. Settings → Collaborators → Add people: B.
B	Accept the invitation (e-mail, or `github.com/A/pr-practice/invitations`).
B	`git clone git@github.com:A/pr-practice.git practice/hw7_pr`, then `cd practice/hw7_pr`.
B	`git switch -c add-b-note`, create `notes-from-b.md`, `git add`, `git commit`, `git push -u origin add-b-note`.
B	On GitHub: *Compare & pull request*, title and description, *Create pull request*.
A	*Files changed*: inline comment with a suggestion; *Review changes* → *Request changes* → *Submit review*.

Who	Step
B	*Commit suggestion* (or fix locally: `git commit`, `git push`).
A	*Approve*; *Create a merge commit*; confirm.
B	*Delete branch*; locally `git switch main`, `git pull`, `git branch -d add-b-note`.
B	`printf 'my-pr=A/pr-practice#1\n' > ../hw7_pr_answers.txt` (from inside `hw7_pr`).

Round 2: fork (A opens the pull request).

Who	Step
B	Create a *public* repository `pr-practice-b` with a README.
A	*Fork* the repository, then `git clone git@github.com:A/pr-practice-b.git practice/hw7_pr`.
A	`git remote add upstream git@github.com:B/pr-practice-b.git`; `git remote -v` shows both remotes.
A	`git switch -c add-a-note`, add a file, commit, `git push -u origin add-a-note`.
A	On GitHub open the pull request; check that the base repository is `B/pr-practice-b`.
B	Review with a suggestion and *Request changes*.
A	*Commit suggestion*.
B	*Approve*; *Squash and merge*.
A	`git fetch upstream`, `git switch main`, `git merge upstream/main`, `git push origin main`.
A	`printf 'my-pr=B/pr-practice-b#1\n' > ../hw7_pr_answers.txt`.

In Round 1, A has no local clone, and in Round 2, B has none; the names and the numbers `#1` above are placeholders. Each of you writes the pull request *you opened*: B the one from Round 1, A the one from Round 2. Both repositories must be public, because the checker asks GitHub's public service whether the pull request is merged. If the line is missing the checker skips nothing silently: a missing or private pull request is reported as a failure, while an unreachable GitHub (no network, or more than 60 anonymous requests an hour) skips only that one check. The `upstream` check applies only to the fork route (A). After Round 2, A's `practice/hw7_pr` is the fork clone, so `origin` points at A's fork on github.com and `upstream` at B's repository.

Common mistakes.

- Merging with a fast-forward in 7.4 (`git merge teammate-feature` without `--no-ff`): the server's `main` then has no merge commit and the checker fails 7.4. Reset and redo.
- Opening the pull request *to your own fork*. Check that the base repository is the original owner's.
- B not accepting the collaborator invitation: the push in Round 1 fails with "Permission denied".
- Creating a repository as *Private*: the checker reports "GitHub cannot find ...".
- Forgetting `upstream`, then wondering why `git fetch upstream` fails ("not a git repository" or "no such remote").
- Writing the pull request number of your *partner's* request, or a private repository, in `hw7_pr_answers.txt`.
- `git branch -d` refusing after *Squash and merge*: expected (see the lecture); if the pull request really is merged, `git branch -D` is fine.